

1.1 区间和

...

只要找 k 两边的索引，单调栈通过弹出操作来快速找到左右两边所要的第一个大于等于 $a[k]$ 的索引，在寻找的过程中记录下区间内最小的前缀和，以及最大的后缀和，以进行最后判断。

栈顶到栈底递增

Algorithm MaxSumCheck($a[0..n-1]$):

Function solve(Array a):

$n \leftarrow \text{length of } a$

If $n = 0$: Return "YES"

// 1. 计算前缀和 $P[0..n]$ ，其中 $P[0] = 0$, $P[i] = a[0] + \dots + a[i-1]$

$P[0] \leftarrow 0$

For i from 0 to $n-1$:

$P[i+1] \leftarrow P[i] + a[i]$

// 2. 从左到右: 计算每个位置 k 作为最大值时，左侧可选的最小前缀和

// $\text{minLeft}[k] = \min\{ P[i] \mid i \in (L, k] \}$ ，其中 L 是 k 左侧第一个 $\geq a[k]$ 的位置

Initialize empty stack $S1$ // 存储 (index, min_prefix_value)

For k from 0 to $n-1$: //左侧

$\text{currentMin} \leftarrow P[k]$

while $S1$ is not empty and $a[S1.\text{top}().\text{index}] \leq a[k]$:

$\text{popped} \leftarrow S1.\text{pop}()$

$\text{currentMin} \leftarrow \min(\text{currentMin}, \text{popped.min_prefix_value})$ //把弹出元素的值记

录下来

$\text{minLeft}[k] \leftarrow \text{currentMin}$

$S1.\text{push}(k, \text{currentMin})$

// 3. 从右到左: 计算每个位置 k 作为最大值时，右侧可选的最大前缀和

// $\text{maxRight}[k] = \max\{ P[j+1] \mid j \in [k, R) \}$ ，其中 R 是 k 右侧第一个 $> a[k]$ 的位

置

Initialize empty stack $S2$ // 存储 (index, max_prefix_value)

For k from $n-1$ down to 0: //右侧

$\text{currentMax} \leftarrow P[k+1]$

while $S2$ is not empty and $a[S2.\text{top}().\text{index}] < a[k]$:

$\text{popped} \leftarrow S2.\text{pop}()$

$\text{currentMax} \leftarrow \max(\text{currentMax}, \text{popped.max_prefix_value})$ //把弹出元素的值记

录下来

$\text{maxRight}[k] \leftarrow \text{currentMax}$

$S2.\text{push}(k, \text{currentMax})$

// 4. 检查是否存在以 $a[k]$ 为最大值的子数组，其和 $> a[k]$

For k from 0 to $n-1$:

If $\text{maxRight}[k] - \text{minLeft}[k] > a[k]$:

Return "NO"

Return "YES"

Main program:

Read number of test cases t

For each test case:

```
Read n
Read array a of n integers
Print solve(a)
```

时空复杂度:

指标	复杂度	说明
时间复杂度	$O(n)$	每个元素入栈出栈常数次数, 三次线性遍历
空间复杂度	$O(n)$	存储前缀和、minLeft、maxRight 和栈

'''

```
import sys

def solve(a):
    n = len(a)
    if n == 0:
        return "YES"

    # 1. 前缀和 P[0..n], P[i] = sum(a[0:i])
    P = [0] * (n + 1)
    for i in range(n):
        P[i + 1] = P[i] + a[i]

    # 2. 计算 min_P_left[k] = min{ P[i] | i ∈ (L, k] }
    min_P_left = [0] * n
    stack = [] # (index, min_prefix_value)
    for k in range(n):
        current_min = P[k] # 对应子数组从 k 开始 (i = k)
        # 弹出所有 a[stack_top] <= a[k] 的元素 (找左边第一个 > a[k] 的位置)
        while stack and a[stack[-1][0]] <= a[k]:
            _, prev_min = stack.pop()
            if prev_min < current_min:
                current_min = prev_min
        min_P_left[k] = current_min
        stack.append((k, current_min))

    # 3. 计算 max_P_right[k] = max{ P[j+1] | j ∈ [k, R) }
    max_P_right = [0] * n
    stack = [] # (index, max_prefix_value)
    for k in range(n - 1, -1, -1):
        current_max = P[k + 1] # 对应子数组到 k 结束 (j = k)
        # 弹出所有 a[stack_top] < a[k] 的元素 (注意是 <, 避免重复)
        while stack and a[stack[-1][0]] < a[k]:
            _, prev_max = stack.pop()
            if prev_max > current_max:
                current_max = prev_max
        max_P_right[k] = current_max
        stack.append((k, current_max))

    # 4. 检查是否存在违规子数组
    for k in range(n):
        if max_P_right[k] - min_P_left[k] > a[k]:
            return "NO"
    return "YES"
```

```
# ===== 主程序保持不变 =====
if __name__ == "__main__":
    input = sys.stdin.readline
    t = int(input())
    results = []
    for _ in range(t):
        n = int(input())
        a = list(map(int, input().split()))
        results.append(solve(a))
    for res in results:
        print(res)
```

1.2稠密有向无环图

```
'''
Function solve(l, r):
    If l ≥ r:
        Return

    mid ← (l + r) // 2

    // 左区间连接到中点
    For i from l to mid-1:
        G.append((i, mid))

    // 中点连接到右区间
    For i from mid+1 to r:
        G.append((mid, i))

    // 递归处理左右两部分
    solve(l, mid-1)
    solve(mid+1, r)

Main program:
    G ← empty list
    Read n
    solve(1, n)
    Output size of G
    Output all edges in G
```

复杂度:
 时间复杂度 $O(n \log n)$
 空间复杂度 $O(n \log n)$

```
'''

def solve(l, r):
    if l >= r:
        return
    mid = (l + r) // 2
    for i in range(l, mid):
```

```

        G.append((i, mid))
    for i in range(mid + 1, r + 1):
        G.append((mid, i))
    solve(l, mid - 1)
    solve(mid + 1, r)

G = []
n = int(input())
solve(1, n)
print(len(G))
for u, v in G:
    print(u, v)

```

1.3 能耗划分

```

'''
算法：计算区间最小能耗（分治递归）

Function solve(L, R, points, l, r, A, B):
    // 统计区间内点数
    cnt ← r - l
    If cnt = 0:
        Return A

    length ← R - L + 1          // 区间长度
    cost_whole ← B × cnt × length // 直接处理的代价

    // 区间长度为1，无法再分割
    If L = R:
        Return cost_whole

    // 分割目标点到左右区间，使得points[l:m1]在左区间，points[m1:r]在右区间
    mid ← (L + R) // 2
    m1 ← l
    while m1 < r AND points[m1] ≤ mid:
        m1 ← m1 + 1

    // 递归处理左右子区间
    res_left ← solve(L, mid, points, l, m1, A, B)
    res_right ← solve(mid+1, R, points, m1, r, A, B)

    // 返回最优方案
    Return min(cost_whole, res_left + res_right)

Main program:
    Read n, k, A, B
    Read points[0...k-1]
    Sort points
    total ← 2^n
    Output solve(1, total, points, 0, k, A, B)

```

复杂度：

递归深度： $O(n)$ 每层的工作量： $O(k)$

总时间复杂度: $O(n \times k)$

递归栈空间: $O(n)$ 数组存储: $O(k)$

总空间复杂度: $O(n + k)$

问题: 错误代码中区间里的 k 个点要重复定位

```
...

import sys

sys.setrecursionlimit(1 << 25)

def solve(L, R, points, l, r, A, B):
    cnt = r - l # points[l:r]均在[L,R]内
    if cnt == 0:
        return A
    length = R - L + 1
    cost_whole = B * cnt * length
    if L == R:
        return cost_whole
    mid = (L + R) // 2
    # 寻找左右子区间的下标范围
    m1 = l
    m2 = r
    while m1 < m2 and points[m1] <= mid:
        m1 += 1
    res_left = solve(L, mid, points, l, m1, A, B)
    res_right = solve(mid + 1, R, points, m1, r, A, B)
    return min(cost_whole, res_left + res_right)

def main():
    n, k, A, B = map(int, sys.stdin.readline().split())
    points = list(map(int, sys.stdin.readline().split()))
    points.sort()
    total = 1 << n
    print(solve(1, total, points, 0, k, A, B))

if __name__ == "__main__":
    main()
```

1.4 中间数

```
...

算法: 计算区间  $[1, n]$  最终集合  $S$  的元素个数 (分治递归 + 记忆化)

FUNCTION dfs(r):
    IF r IN memo THEN:
        RETURN memo[r] // 直接返回缓存结果

    // 边界条件
    IF r = 1 THEN:
        RETURN 1
    IF r = 2 THEN:
        RETURN 1
```

```

IF r = 3 THEN:
    RETURN 2

// 计算中点（向上取整）
mid ← (r + 1) >> 1

// 根据奇偶性分情况讨论
IF r % 2 = 0 THEN:
    // 偶数长度：区间不对称
    result ← dfs(mid) + dfs(mid + 1) - 1
ELSE:
    // 奇数长度：区间对称
    result ← dfs(mid) * 2 - 1

// 存入缓存
memo[r] ← result
RETURN result

```

主程序：

```

READ n
memo ← {} // 初始化空字典
PRINT dfs(n)

```

复杂度分析：

递归深度： $O(\log n)$ 每层的工作量： $O(1)$

总时间复杂度： $O(\log n)$

递归栈空间： $O(\log n)$ 缓存存储： $O(\log n)$

总空间复杂度： $O(\log n)$

问题：相同dfs子问题会被重复计算，需要加缓存
...

```

def solve():
    import sys
    sys.setrecursionlimit(1 << 25)

    n = int(sys.stdin.readline())

    memo = {}

    def dfs(r):
        if r in memo:
            return memo[r]
        if r == 1:
            return 1
        if r == 2:
            return 1
        if r == 3:
            return 2

        mid = (r + 1) >> 1
        if r % 2 == 0:
            result = dfs(mid) + dfs(mid + 1) - 1 #mid+1其实是mid到r的长度

```

```

        else:
            result = dfs(mid) * 2 - 1

        memo[r] = result
        return result

    print(dfs(n))

if __name__ == "__main__":
    solve()

```

2.1 前缀和不少于0

```

'''
前缀和为0和括号配对、出栈序列问题思想一样

$$C(2n, n) = [(2n) \times (2n-1) \times (2n-2) \times \dots \times (n+1)] / [n \times (n-1) \times (n-2) \times \dots \times 1]$$


$$C(2n, n) = [(2n) / 1] \times [(2n-1) / 2] \times [(2n-2) / 3] \times \dots \times [(n+1) / n]$$


Function main():
    Read n

    If n == 0:
        Output 0
        Return

    // 计算组合数 C(2n, n)
    result = 1
    For i from 1 to n:
        result = result * (2 * n - i + 1) // i

    // 计算卡特兰数 C_n = C(2n, n) / (n + 1)
    catalan = result // (n + 1)
    Output catalan

Main program:
    main()

复杂度分析:
时间复杂度: O(n)
空间复杂度: O(1)
'''

def main():
    n = int(input().strip())

    if n == 0:
        print(0)
        return

    # 计算组合数 C(2n, n)
    result = 1
    for i in range(1, n + 1):

```

```

        result = result * (2 * n - i + 1) // i

# 计算卡特兰数 C_n = C(2n, n) / (n + 1)
catalan = result // (n + 1)
print(catalan)

if __name__ == "__main__":
    main()

```

2.2 最小最大值

...

朴素dp: $f[i]=f[j]+\max(a[j+1]\dots a[i])(s[i]-s[j]\leq m)$ $f[i]$ 为当前层最小代价

优化点: 单调队列 + 双栈 优化dp

1. 单调队列的意义是快速找到一层中合适的分段点

2. 双栈的意义是快速找到一层中所有分段点策略下的最小f, 以及利用重构解决一点空间复杂度

伪代码:

Algorithm minmax(n, m, h[1..n]):

Function main():

 Read n, m

 Read height array h[1..n]

Initialize data structure:

 f[n+2] //dp数组, $f[i]$ 表示前i本书的最小代价

 tmpf[n+2] //存储队列中对应位置的临时f值

 q[n+2] //单调队列, 存储索引, $h[q]$ 递减

 l, r //队列左右指针

 stk[2][n+2] //双栈结构(左栈0, 右栈1)

 top[2] //两个栈的栈顶指针

 st //合法起始位置

 sum_h //辅助高度和

 mid //队列中点, 用于分割左右栈

push(x, i):

 //将x压入栈i, i为栈编号0或1

 if top[i] == 0 or tmpf[stk[i][top[i]]] > tmpf[x]:

 stk[i][top[i]] = x //覆盖值至当前位置, 后续弹出操作只需更改栈顶指针top

 top[i]++

Function rebuild():

 mid ← floor((l+r)/2)

 clear top[0], top[1]

 // 重建左右栈

 for i from mid down to l: push(i, 0)

 for i from mid+1 to r: push(i, 1)

for i from 1 to n:

 //维护合法起点st

 sum_h += h[i]

 while sum_h > m:

 sum_h -= h[st]

 st++


```

// 维护单调队列
while l ≤ r 且 h[q[r]] ≤ h[i]:
    // 弹出队尾元素
    if top[0] > 0 and stk[0][top[0]] == r:
        top[0] -= 1
    if top[1] > 0 and stk[1][top[1]] == r:
        top[1] -= 1
    r--
// 若队尾越过中点，重建栈
if r ≤ mid:
    Rebuild()

// 先更新tmpf, 后r++
if l > r: tmpf[r+1] = f[st-1] + h[i]
    else: tmpf[r+1] = f[q[r]] + h[i]

// 当前书加入队列
r++, q[r] = i, push(r, 1)

// 处理过期队头
while l ≤ r 且 q[l] < st:
    // 若队头在栈顶，弹出
    if top[0] > 0 and stk[0][top[0]] == l:
        top[0] -= 1
    if top[1] > 0 and stk[1][top[1]] == l:
        top[1] -= 1
    l++
// 若队头越过中点，重建栈
if l > mid:
    Rebuild()

// 计算f[i]
f[i] = min(
    f[st-1] + h[q[l]],          // 整段策略
    tmpf[stk[0][top[0]]],      // 左栈策略
    tmpf[stk[1][top[1]]]       // 右栈策略
)

```

```
print(f[n])
```

Main program:

```
main()
```

时间复杂度: $O(n)$ 每本书处理一次, 队列和栈操作均摊 $O(1)$

空间复杂度: $O(n)$ 用于存储 `dp` 数组和辅助结构

```
'''
```

```
import sys
```

```
def main():
```

```
    n, m = map(int, sys.stdin.readline().split())
```

```
    h = [0] * (n + 2) # 1-based索引存储书的高度
```

```
    for i in range(1, n + 1):
```

```
        h[i] = int(sys.stdin.readline())
```

```

# 初始化数据结构
q = [0] * (n + 2) # 单调队列，存储书的索引
stk = [[0] * (n + 2) for _ in range(2)] # 两个栈：左栈(0)和右栈(1)
top = [0, 0] # 栈顶指针
l, r = 1, 0 # 队列的左右指针
mid = 0 # 队列中点，用于分割左右栈
f = [0] * (n + 2) # dp数组，f[i]表示前i本书的最小总代价
tmpf = [0] * (n + 2) # 存储队列中对应位置的临时f值

st = 1 # 当前层的起始位置
sum_h = 0 # 当前层的高度和

def push(x, i):
    """将x推入栈i(0或1)，维护栈的单调性"""
    nonlocal top
    if top[i] == 0:
        top[i] += 1
        stk[i][top[i]] = x
    else:
        if tmpf[stk[i][top[i]]] > tmpf[x]:
            top[i] += 1
            stk[i][top[i]] = x

def rebuild():
    """重建左右栈，根据当前队列的中点分割"""
    nonlocal mid, top
    mid = (l + r) // 2
    top[0] = top[1] = 0
    # 左栈：从mid到l倒序推入
    for i in range(mid, l - 1, -1):
        push(i, 0)
    # 右栈：从mid+1到r正序推入
    for i in range(mid + 1, r + 1):
        push(i, 1)

for i in range(1, n + 1):
    sum_h += h[i]
    # 调整当前层的起始位置，确保和不超过m
    while sum_h > m:
        sum_h -= h[st]
        st += 1

    # 维护单调队列：确保队列中元素的h值递减
    while l <= r and h[q[r]] <= h[i]:
        # 弹出队尾元素
        if top[0] > 0 and stk[0][top[0]] == r:
            top[0] -= 1
        if top[1] > 0 and stk[1][top[1]] == r:
            top[1] -= 1
        r -= 1
    # 若队尾越过中点，重建栈
    if r <= mid:
        rebuild()

```

```

# 计算新队尾的临时f值
if l > r:
    tmpf[r + 1] = f[st - 1] + h[i]
else:
    tmpf[r + 1] = f[q[r]] + h[i] #必要要从上一个最大点开始，不然往后考虑没意义
# 将当前书加入队列
r += 1
q[r] = i
push(r, 1) # 推入右栈

# 处理队头过期元素（不在合法范围内）
# 检查栈顶是否为队头，是的话弹出
if top[0] > 0 and stk[0][top[0]] == 1:
    top[0] -= 1
if top[1] > 0 and stk[1][top[1]] == 1:
    top[1] -= 1

# 弹出所有过期的队头元素
while l <= r and q[l] < st:
    l += 1
    # 若队头越过中点，重建栈
    if l > mid:
        rebuild()
    # 再次检查栈顶是否为新队头
    if top[0] > 0 and stk[0][top[0]] == 1:
        top[0] -= 1
    if top[1] > 0 and stk[1][top[1]] == 1:
        top[1] -= 1

# 计算当前f[i]的最小值
f[i] = f[st - 1] + h[q[l]] # 从起始位置到当前的段
# 比较左右栈中的最小值
if top[0] > 0:
    f[i] = min(f[i], tmpf[stk[0][top[0]]])
if top[1] > 0:
    f[i] = min(f[i], tmpf[stk[1][top[1]]])

print(f[n])

if __name__ == "__main__":
    main()

```

2.3 权重路径方案

```

...
Function main():
    Read n, m
    Read matrix a[1..n][1..m]

    // 正向DP: 预处理从(1,1)到各点的最大路径和
    f[1][1] = a[1][1]
    For i from 1 to n:
        For j from 1 to m:
            If (i,j) ≠ (1,1):
                f[i][j] = max(f[i-1][j], f[i][j-1]) + a[i][j]

```

```

// 逆向DP: 预处理从各点到(n,m)的最大路径和
g[n][m] = a[n][m]
For i from n to 1:
    For j from m to 1:
        If (i,j) ≠ (n,m):
            g[i][j] = max(g[i+1][j], g[i][j+1]) + a[i][j]

// 找出所有在最大路径上的点
points = []
For i from 1 to n:
    For j from 1 to m:
        If f[i][j] + g[i][j] - a[i][j] == f[n][m]:
            points.append((i, j))

// 对路径上的每个点尝试设为0并重新计算最大路径和
res = ∞
For p in points:
    px, py = p
    original = a[px][py]
    a[px][py] = 0 // 暂时设为0

    // 重新计算最大路径和
    dp[1][1] = a[1][1]
    For i from 1 to n:
        For j from 1 to m:
            If (i,j) ≠ (1,1):
                dp[i][j] = max(dp[i-1][j], dp[i][j-1]) + a[i][j]

    res = min(res, dp[n][m])
    a[px][py] = original // 恢复原值

```

Output res

Main program:

```
main()
```

复杂度:

时间复杂度: $O(nm(n+m))$

空间复杂度: $O(nm)$

'''

```
import sys
```

```
def main():
```

```

    n, m = map(int, input().split())
    a = [[0] * (m + 2) for _ in range(n + 2)]
    for i in range(1, n+1):
        row = list(map(int, input().split()))
        for j in range(1, m+1):
            a[i][j] = row[j-1]

```

```

# 正向DP: f[i][j] 从(1,1)到(i,j)的最大路径和
f = [[-10**18]*(m+2) for _ in range(n+2)]

```

```

f[1][1] = a[1][1]
for i in range(1, n+1):
    for j in range(1, m+1):
        if i == 1 and j == 1:
            continue
        f[i][j] = max(f[i-1][j], f[i][j-1]) + a[i][j]

# 逆向DP: g[i][j] 从(i,j)到(n,m)的最大路径和
g = [[-10**18]*(m+2) for _ in range(n+2)]
g[n][m] = a[n][m]
for i in range(n, 0, -1):
    for j in range(m, 0, -1):
        if i == n and j == m:
            continue
        g[i][j] = max(g[i+1][j], g[i][j+1]) + a[i][j]

max_path = f[n][m]
path_points = []
# 找出所有满足最大路径条件的点: 在一条最大路径上的点
for i in range(1, n+1):
    for j in range(1, m+1):
        if f[i][j] + g[i][j] - a[i][j] == max_path:
            path_points.append((i, j))

res = float('inf')
for px, py in path_points:
    # 暂时将 a[px][py] := 0
    original = a[px][py]
    a[px][py] = 0

    # 再做一次DP求最大路径
    dp = [[-10**18]*(m+2) for _ in range(n+2)]
    dp[1][1] = a[1][1]
    for i in range(1, n+1):
        for j in range(1, m+1):
            if i == 1 and j == 1:
                continue
            dp[i][j] = max(dp[i-1][j], dp[i][j-1]) + a[i][j]
    res = min(res, dp[n][m])
    # 恢复
    a[px][py] = original

print(res)

if __name__ == "__main__":
    main()

```

2.4 合法h序列

```

/*
g[i][j] 的含义是前 i 头牛满足以下状态的方案数:
1. 前 i-1 头牛饥饿值为 0 (注意: 在减去偏移量 d 也就是最终饥饿值的意义下);
2. 第 i 头牛饥饿值为 j。

```

```

Function solve():
    Read n, a[1..n]
    g[n+1][max(a)+1]=0
    mn ← min(a[1..n])

    //n为偶数时，d为多少没意义，因为答案是初始状态
    If n % 2 == 0:
        mn ← 0

    ans ← 0

    // 枚举所有可能的偏移量d（最终的饥饿值）
    For d from 0 to mn:
        // 初始化边界条件
        For i from 0 to a[1]:
            g[1][i] = i + 1

        // 动态规划转移
        For i from 2 to n:
            // 计算 g[i][0]
            If min(a[i], a[i-1]) - d ≥ 0:
                g[i][0] ← g[i-1][min(a[i], a[i-1]) - d]

            // 使用前缀和优化计算 g[i][j]
            For j from 1 to a[i] - d:
                If min(a[i] - j, a[i-1]) - d ≥ 0:
                    g[i][j] ← (g[i][j-1] + g[i-1][min(a[i] - j, a[i-1]) - d])
                    //i-1的方案j'有要求，要满足j'≤a[i-1],又要满足j+j'≤a[i],因为要喂
                    饱第i-1头牛
                Else:
                    g[i][j] ← g[i][j-1]

            ans ← (ans + g[n][0])

    Output ans

Main program:
    solve()

```

复杂度分析：

可以通过处理前缀和的方式 $O(1)$ 转移，DP 复杂度 $O(NH)$ ，其中 H 是数组 a 中的最大值。

时间复杂度 $O(NH^2)$

空间复杂度 $O(NH)$

*/

```

#include<algorithm>
#include<cstdio>
#include<cstring>
#include<iostream>
#include<queue>
#include<vector>
using namespace std;
inline int mmin(register int x, register int y) {
    return x > y ? y : x;
}

```

```

const long long mod = 1000000007;
int n;
int a[107], mn = 1000;
long long ans, g[107][1007];
signed main() {
    scanf("%d", &n);
    for(int i = 1; i <= n; ++i) {
        scanf("%d", a + i);
        mn = min(mn, a[i]);
    }
    for(int i = 0; i <= a[1]; ++i) g[1][i] = i + 1; //这里省略了求和
    if(n % 2 == 0) mn = 0;
    for(int d = 0; d <= mn; ++d) {
        for(int i = 2; i <= n; ++i) {
            g[i][0] = g[i - 1][min(a[i], a[i - 1]) - d];
            for(int j = 1; j <= a[i] - d; ++j) {
                g[i][j] = g[i][j - 1] + g[i - 1][min(a[i] - j, a[i - 1]) - d];
                if(g[i][j] >= mod) g[i][j] -= mod;
            }
        }
        ans += g[n][0]; if(ans >= mod) ans -= mod;
    }
    printf("%lld\n", ans);
    return 0;
}

```

3.1 颜色石头

```

...
Function solve():
    Read inputs k, p, q and counts array
    Convert p, q to 0-based indexing

    // 基础检查: 起点和终点的石头必须存在
    If counts[p] == 0 or counts[q] == 0:
        Return 0

    s = sum(counts) // 总石头数量
    max_count = max(counts) // 最大颜色数量

    // 快速筛选: 如果最大数量满足最严格限制
    If 2 * max_count <= s - 1:
        Return 1

    // 详细检查: 对每种最大数量的颜色进行验证
    For each color i, when counts[i] == max_count:
        bonus = 0
        If i == p: bonus += 1 // 起点颜色奖励
        If i == q: bonus += 1 // 终点颜色奖励

        // 核心条件: 2 * count <= s + bonus - 1
        If 2 * count > s + bonus - 1:
            Return 0

    Return 1

```

```
Main program:
    solve()
```

复杂度分析:

时间复杂度 $O(n)$

空间复杂度 $O(1)$

'''

```
import sys
```

```
def solve():
```

```
    # 使用 sys.stdin.read 一次性读取所有输入，应对大数据量
    input_data = sys.stdin.read().split()
```

```
    if not input_data:
        return
```

```
    iterator = iter(input_data)
```

```
    try:
```

```
        k = int(next(iterator)) # k: 石头种类数量
        p = int(next(iterator)) # p: 第一块石头颜色
        q = int(next(iterator)) # q: 最后一块石头颜色
```

```
        # 读取每种颜色的数量
```

```
        counts = [int(next(iterator)) for _ in range(k)]
```

```
    except StopIteration:
```

```
        return
```

```
    # 输入的 p, q 是 1-based 索引，转换为 0-based
```

```
    p -= 1
```

```
    q -= 1
```

```
    # 1. 基础检查：起点和终点的石头必须存在
```

```
    if counts[p] == 0 or counts[q] == 0:
        print(0)
        return
```

```
    s = sum(counts)
```

```
    max_count = max(counts)
```

```
    # 2. 快速筛选
```

```
    # 最严格的限制是当某种颜色既不是起点也不是终点时:
```

```
    # 需要满足  $2 * \text{count} \leq s - 1$ 
```

```
    if 2 * max_count <= s - 1:
        print(1)
        return
```

```
    # 3. 详细检查
```

```
    for i, count in enumerate(counts):
        if count == max_count:
            bonus = 0
            if i == p:
```



```

        bonus += 1
    if i == q:
        bonus += 1

    # 核心公式:  $2 * \text{count} \leq s + \text{bonus} - 1$ 
    # 变形为判断失败条件:  $2 * \text{count} > s + \text{bonus} - 1$ 
    if  $2 * \text{count} > s + \text{bonus} - 1$ :
        print(0)
        return

    # 如果所有高频颜色都满足条件
    print(1)

if __name__ == '__main__':
    solve()

```

3.2 最小环覆盖

```

/*
Algorithm Circular_Cover_MinIntervals(n, k, intervals[1..k])

// Step 1: 展开环为线性 [1, 2n]
for i ← 1 to k do
    L ← intervals[i].first
    R ← intervals[i].second
    if L > R then
        R ← R + n                // 跨越环起点, 展开到第二圈
    a[i] ← (L, R)

// Step 2: 按左端点排序
sort a[1..k] by L in non-decreasing order

// Step 3: 预处理 f[i][0] -- 从位置 i 出发, 用一个区间最远能跳到的位置 (开区间)
now ← 1                        // 当前考虑的区间索引
r ← 0                          // 当前能覆盖到的最右边界 (开区间)
for i ← 1 to 2·n do
    while now ≤ k and a[now].first ≤ i do
        r ← max(r, a[now].second + 1)
        now ← now + 1
    f[i][0] ← r

// Step 4: 倍增预处理 f[i][j] = f[f[i][j-1]][j-1], f[i][j]: 从位置 i 出发, 用 2^j 个
// 区间, 最远能跳到哪里。
for j ← 1 to 20 do
    for i ← 1 to 2·n do
        f[i][j] ← f[ f[i][j-1] ][j-1] // 这里很精妙, 代表 f[i][j-1] 再跳 2^(j-1) 次到达 f[i][j]

// Step 5: 枚举所有可能的起点 i ∈ [1, 2n], 尝试覆盖 [i, i+n)
ans ← ∞
for i ← 1 to 2·n do
    x ← i
    ret ← 0

```

```

// 贪心倍增：尽可能跳远但不超过 i + n
for j ← 20 downto 0 do
    if f[x][j] - i < n then
        x ← f[x][j]
        ret ← ret + 2^j // 13 = 8 + 4 + 1 = 2^3 + 2^2 + 2^0

// 再跳一步完成覆盖
x ← f[x][0]
ret ← ret + 1

// 检查是否成功覆盖至少 n 长度
if x - i ≥ n then
    ans ← min(ans, ret)

// Step 6: 输出结果
if ans == ∞ then
    output "impossible"
else
    output ans

```

复杂度分析：

时间复杂度：O((n + k) log n)

空间复杂度：O(n log n)

*/

```

#include<bits/stdc++.h>
#define fi first
#define se second
using namespace std;
const int N=2e6+5;
int n,k,f[N][30],ans=1e9;
pair<int,int> a[N];
int main(){
    scanf("%d%d",&n,&k);
    for(int i=1;i<=k;i++){
        scanf("%d%d",&a[i].fi,&a[i].se);
        if(a[i].fi>a[i].se) a[i].se+=n;
    }
    sort(a+1,a+1+k);
    int now=1,r=0;
    for(int i=1;i<=2*n;i++){
        while(now<=k&&a[now].fi<=i){
            r=max(r,a[now].se+1);
            now++;
        }
        f[i][0]=r;
    }//预处理跳一次跳到哪
    for(int j=1;j<=20;j++){
        for(int i=1;i<=2*n;i++){
            f[i][j]=f[f[i][j-1]][j-1];
        }
    }//倍增预处理
    for(int i=1;i<=2*n;i++){
        int x=i,ret=0;
        for(int j=20;j>=0;j--){

```

```

        if(f[x][j]-i<n){
            x=f[x][j];
            ret+=1<<j;
        }
    }
    x=f[x][0];ret++;
    if(x-i>=n) ans=min(ans,ret);
} //枚举左端点再跑
if(ans==1e9) printf("impossible\n");
else printf("%d\n",ans);
return 0;
}

```

3.3 012消消乐

```

/*

Function solve(s)
n = length of s

// 第一步：对偶数位置(1-based)的字符进行处理
For i = 1 to n-1 with step 2 (positions 2, 4, 6... in original string)
    if s[i] == '0', then s[i] = '1'
    else if s[i] == '1', then s[i] = '0' // '2' unchanged

// 第二步：统计字符 0, 1, 2 的数量
cnt[0] = count of character '0'
cnt[1] = count of character '1'
cnt[2] = count of character '2'

// 第三步：枚举将多少个 '2' 转换为 '0'
ans = ∞
For i from 0 to cnt[2]
    c0 = cnt[0] + i // 最终 '0' 的数量
    c1 = cnt[1] + (cnt[2] - i) // 最终 '1' 的数量
    cur = |c0 - c1| // 两者差值的绝对值
    If cur < ans, then ans = cur

return ans

复杂度分析：
时间复杂度 O(n * cnt[2])
空间复杂度 O(1)
*/

#include <iostream>
#include <string>
#include <vector>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

```

```

string s;
if (!(cin >> s)) return 0;
int n = s.size();

// 第一步: 偶数位置 (1-based) 0/1 取反, 2 不动
// s 是 0-based, 所以 i = 1, 3, 5, ... 对应 2,4,6,... 位置
for (int i = 1; i < n; i += 2) {
    if (s[i] == '0') s[i] = '1';
    else if (s[i] == '1') s[i] = '0';
    // s[i] == '2' 时不改
}

// 第二步: 统计 0,1,2 个数
long long cnt[3] = { 0, 0, 0 };
for (char c : s) {
    if (c == '0') cnt[0]++;
    else if (c == '1') cnt[1]++;
    else if (c == '2') cnt[2]++;
}

// 第三步: 枚举有多少个 2 变成 0, 其余变成 1
long long ans = (long long)1e18;
for (long long i = 0; i <= cnt[2]; ++i) {
    long long c0 = cnt[0] + i;
    long long c1 = cnt[1] + (cnt[2] - i);
    long long cur = llabs(c0 - c1);
    if (cur < ans) ans = cur;
}

cout << ans << '\n';
return 0;
}

```

3.4 魔法物品

```

/*
Algorithm magic

//读取输入
read n,p
read n items (a,b)
// p: magic scroll price
// a: initial money from item
// b: money after identifying item (if no magic, b = a)
allv = Σa // total initial money
initial_money = 0

// 1. 分类物品
for each item (a, b):
    if b exists AND b - a > p:
        magic_items.add( v=a, profit=b-a-p )
    else:
        initial_money += a

// 2. 判断

```

```

if initial_money ≥ p:
    // 全部鉴定
    answer = Σa + Σprofit
else:
    // 0-1背包求最小损失
    // f[j] = 凑够价格j的最小损失
    f[0] = 0, f[others] = ∞

    for each magic_item:
        for j from allv down to item.v:
            if j ≥ item.v:
                f[j] = min(f[j], f[j-item.v] + item.profit)

    // 找最小损失 (需要凑够 p - initial_money)
    min_loss = min{ f[j] | j ≥ p - initial_money }

    answer = Σa + Σprofit - min_loss

    if answer ≤ allv:
        answer = allv // 只能全卖

```

复杂度:

时间复杂度: $O(n * allv)$

空间复杂度: $O(allv)$

*/

```

#include<iostream>
#include<cstdio>
#include<sstream>
#include<cstring>
#include<algorithm>
using namespace std;
int f[10000005];
int n,p; // 物品总数,魔法卷轴价格
int th_len; // 魔法物品的个数
struct thing {
    int v; //初始资金
    int p; //魔法卷轴价格
    int t; //损失差
}th[1005]; //魔法物品数组
int main(){
    freopen("in.txt","r",stdin);
    //freopen("out.txt","w",stdout);
    string in;
    cin >> n >> p;
    getline(cin,in); // 读取换行
    int v = 0,allv = 0,callv;
    for(int i = 1;i <= n;i++){
        stringstream st;
        getline(cin,in);
        st << in;
        int a,b; //基础价格或基础价格加魔法价格
    }
}

```

```

    st >> a;
    allv += a;
    if(st >> b && b - a > p){ // st >> b 读取成功 代表存在魔法属性
        th[++th_len].v = a;
        th[th_len].p = b;
        th[th_len].t = b - a - p; // 损失差，方便下一步的动归求解
    }else
        v += a;
}
callv = allv; //由于接下来会对 allv 进行操作，所以用 callv 保存所有物品的价格和，以便
出现卖出所有魔法物品后仍无法购买卷轴时直接输出
for(int i = 0;i <= th_len;i++){
    for(int j = 1;j <= allv;j++){
        f[j] = 999999999; //通过卖出部分魔法物品凑够价格j的最小损失
    }
}
if(p > v){ //当初始资金无法购买一个魔法卷轴
    for(int i = 1;i <= th_len;i++){
        for(int j = allv;j >= 1;j--){
            f[j] = f[j];
            if(j - th[i].v >= 0){
                f[j] = min(f[j],f[j - th[i].v] + th[i].t);
            }
        }
    }
    int mi = 999999999;
    for(int i = p - v;i <= allv;i++){
        if(f[i] < mi)
            mi = f[i];
    }
    for(int i = 1;i <= th_len;i++)
        allv += th[i].t;
    if(allv - mi <= 0)
        cout << callv;
    else
        cout << allv - mi;
}else{ // 初始资金能够购买一个魔法卷轴
    for(int i = 1;i <= th_len;i++)
        allv += th[i].t;
    cout << allv;
}
return 0;
}

```

4.1 餐巾计划问题

/*

拿到题后根据标签大体上可以判断出这是一个最小费用流。题目中有一段这样说道：

每天结束时，餐厅必须决定将多少块脏的餐巾送到快洗部，多少块餐巾送到慢洗部，以及多少块保存起来延期送洗。

但是每天洗好的餐巾和购买的新餐巾数之和，要满足当天的需求量。

每天结束时，餐厅必须决定将多少块脏的餐巾送到快洗部，多少块餐巾送到慢洗部，以及多少块保存起来延期送洗。

但是每天洗好的餐巾和购买的新餐巾数之和，要满足当天的需求量。

于是整理得，我们需要支持以下操作：

将脏餐巾以 f 元/条送到快洗部，过 m 天后，干净餐巾送回来使用

将脏餐巾以 s 元/条送到慢洗部，过 n 天后，干净餐巾送回来使用

延期送洗（可能会出现之后餐巾需求过少，并不需要所有餐巾的情况）

以 p 元/条购买新的餐巾

于是本题第一个难点就来了：如何处理脏餐巾和干净餐巾？

我们可以想到，每天开始的时候只有干净的餐巾可使用，每天结束的时候仅有脏的餐巾需要操作。于是将每天拆成两个点：起始点与结束点，分别处理不同时间段所需操作。

于是也可以想到：

送到快洗部属于结束点操作，连向 $i+m$ 天后的起始点，费用为 f （表示餐巾洗好了，可使用）

送到慢洗部属于结束点操作，连向 $i+n$ 天后的起始点，费用为 s

延期送洗属于结束点操作，连向 $i+1$ 的结束点，不需费用。

购买新的餐巾的操作也应是连向每天起始点 的边，目前并没有确定从哪连的，但费用为 p 。

以上操作流量均为 inf 。

好了，到这里你有没有发现以上操作有什么共同点？

没错，他们都是连向起始点的有向边！

update:延期送洗的操作应连向的是结束点， 感谢 [Specialzyy](#) 指出错误。

update:话虽如此，但此题没有从起始点向外连的边，仍满足线性。

事实上，网络流24题的全称为“网络流与线性规划24题”，这也就告诉我们： 网络流的建图一定有顺序的，建的边一定是沿源点流向汇点，否则图就会不流通。

所以现在我们弄明白了：我们应该将源点连接每天的结束点，而不是他们的起始点；起始点连向他们的汇点，容量为这天所需的餐巾数量，费用为0。

此外购买餐巾直接从源点购买即可。

之后跑一个裸的费用流即可。

algorithm min_cost_max_flow:

// 输入数据

读取 $n, r[i]$ // 天数和第 i 天的需求量

读取 p, a, f, b, s // p : 购买价; a : 快洗天数; f : 快洗费; b : 慢洗天数; s : 慢洗费

$N = 2 * n + 1$ // 汇点编号

// 初始化图（邻接表）

// 步骤1: 连接源点和汇点的基本需求边

for $i = 1$ to n :

 add_edge(源点 $S=0$, 脏点 i , 容量= $r[i]$, 费用=0) // 每天产生 $r[i]$ 条脏餐巾

 add_edge(干净点 $i+n$, 汇点 $T=N$, 容量= $r[i]$, 费用=0) // 每天必须提供 $r[i]$ 条干净餐巾

// 步骤2: 添加操作边（从脏点出发）

for $i = 1$ to n :

 // 1. 购买新餐巾: 直接供给第 i 天的干净需求

```

add_edge(源点 S=0, 干净点 i+n, 容量=∞, 费用=p)

// 2. 延期送洗: 把第 i 天的脏餐巾留到第 i+1 天处理
if i + 1 <= n:
    add_edge(脏点 i, 脏点 i+1, 容量=∞, 费用=0)

// 3. 快洗: 第 i 天送洗, 第 i+a 天早上可用
if i + a <= n:
    add_edge(脏点 i, 干净点 (i + a) + n, 容量=∞, 费用=f)

// 4. 慢洗: 第 i 天送洗, 第 i+b 天早上可用
if i + b <= n:
    add_edge(脏点 i, 干净点 (i + b) + n, 容量=∞, 费用=s)

// 步骤3: 运行最小费用最大流算法 (SPFA + DFS Dinic)
total_cost = min_cost_max_flow(源点=0, 汇点=N)

// 输出结果
输出 total_cost
*/

#include <iostream>
#include <cstring>
#include <queue>
#define N 2*n+1
#define inf 2147483647
using namespace std;

struct ed{
    int u,next,w,f;
}e[300000];
long long n,p,b,f,a,s,st=1,cost,ans;
int r[100000],d[100000],fir[100000],c[100000];
queue<int> q; bool v[100000];

bool spfa()
{
    for (int i=0;i<=N;i++) d[i]=inf/2,v[i]=0,c[i]=fir[i];
    q.push(0); v[0]=1; d[0]=0;
    while (!q.empty())
    {
        int k=q.front(); q.pop(); v[k]=0;
        for (int i=fir[k];i;i=e[i].next)
        {
            int u=e[i].u,w=e[i].f;
            if (d[u]>d[k]+w&&e[i].w)
            {
                d[u]=d[k]+w; if (!v[u]) v[u]=1,q.push(u);
            }
        }
    }
    return (d[N]<inf/2);
}

```



```

int dfs(int p,int now)
{
    if (p==N) {
        v[N]=1; ans+=now; return now;
    }
    int mw=0,used=0; v[p]=1;
    for (int i=c[p];i;i=e[i].next){
        c[p]=1; int u=e[i].u,w=e[i].f;
        if ((!v[u]||u==N)&&d[u]==d[p]+w&&e[i].w)
            if (mw=dfs(u,min(now-used,e[i].w)))
            {
                e[i].w-=mw; e[i^1].w+=mw; used+=mw;
                cost+=w*mw; if (used==now) break;
            }
    }
    return used;
}

long long dinic()
{
    while (spfa())
    {
        v[N]=1;
        while (v[N])
        {
            memset(v,0,sizeof(v));
            dfs(0,inf);
        }
    }
    return cost;
}

void add(int x,int y,int w,int f)
{
    e[++st].u=y; e[st].next=fir[x]; e[fir[x]=st].w=w; e[st].f=f;
    e[++st].u=x; e[st].next=fir[y]; e[fir[y]=st].w=0; e[st].f=-f;
}

int main()
{
    cin>>n;
    for (int i=1;i<=n;i++) {
        cin>>r[i];
        add(0,i,r[i],0); add(i+n,N,r[i],0);
        //源点连接结束点，起始点连向他们的汇点。
    }
    cin>>p>>a>>f>>b>>s;
    for (int i=1;i<=n;i++)
    {
        add(0,i+n,inf,p); //从源点购买餐巾
        if (i+1<=n) add(i,i+1,inf,0); //把今天的脏毛巾拖到明天
        if (i+a<=n) add(i,i+n+a,inf,f); //把餐巾送到快洗部
        if (i+b<=n) add(i,i+n+b,inf,s); //把餐巾送到慢洗部
    }
}

```

```

}
//可以注意到，以上的连边均是由源点向汇点的。
cout<<dinic()<<endl;
}

```

4.2 最大和乘积

```

/*
// 贪心 + DFS局部搜索

read T, n
sort items by x/y descending // 贪心: x/y越大越该选x
pre_x[i] = sum(x[1..i])      // x前缀和
suf_y[i] = sum(y[i..n])      // y后缀和

// 暴力搜中间部分
for center in [n/2-100, n/2+100]:
    range = [center-7, center+7]
    dfs(pos in range, pre_x[range左], suf_y[range右]):
        if pos > range右: ans = max(ans, now_x * now_y)
        else:
            dfs(pos+1, now_x + x[pos], now_y)      // 选x
            dfs(pos+1, now_x, now_y + y[pos])      // 选y

print ans

*/

#include<bits/stdc++.h> // 包含所有标准库头文件
using namespace std;
#define int __int128 // 将int定义为__int128, 扩大整数范围
const int MAXN=3010, DELTA=100, TEMP=7; // 定义常量: 最大数组大小、搜索范围偏移、临时变量

// 快速读入函数, 用于高效读取整数
inline int read(){
    int x=0, f=1; // x存储结果, f表示正负号
    char ch=getchar(); // 读取字符
    while(ch<'0' || ch>'9'){ // 跳过非数字字符
        if(ch=='-') f=-1; // 处理负号
        ch=getchar();
    }
    while(ch>='0' && ch<='9'){ // 读取数字字符
        x=(x<<3)+(x<<1)+ch-48; // 等价于 x=x*10+(ch-'0')
        ch=getchar();
    }
    return x*f; // 返回结果
}

// 打印函数, 输出__int128类型的数值
inline void print(int x){
    string res;
    while(x) res+=(x%10+'0'), x/=10; // 将数字转换为字符串
    reverse(res.begin(), res.end()); // 反转字符串
}

```

```

        cout<<res<<'\n';    // 输出结果
    }

    // 定义节点结构体, 包含x坐标、y坐标和优先级
    struct Node{
        int x,y;
        double pri;
    }a[MAXN];

    // 全局变量定义
    int T,n,A,B,ans,cntx[MAXN],cnty[MAXN];    // T测试用例数, n节点数, A,B可能的参数, ans结果, 前缀和数组

    // 比较函数, 按优先级降序排列
    inline bool cmp(Node x,Node y){
        return x.pri>y.pri;
    }

    // 深度优先搜索函数
    // pos: 当前位置, ed: 结束位置, nowx: 当前x值, nowy: 当前y值
    void dfs(int pos,int ed,int nowx,int nowy){
        if(pos>ed) ans=max(ans,nowx*nowy);    // 如果到达边界, 更新最大乘积
        else{
            // 分别尝试选择当前节点的x或y值
            dfs(pos+1,ed,nowx+a[pos].x,nowy);    // 选择当前节点的x值
            dfs(pos+1,ed,nowx,nowy+a[pos].y);    // 选择当前节点的y值
        }
    }

    signed main(){
        T=read(),n=read(),A=read(),B=read();    // 读取测试用例数T, 节点数n, 以及A、B参数

        while(T--){    // 对每个测试用例进行处理
            // 读取每个节点的x、y值, 并计算优先级 (x/y的比值)
            for(int i=1;i<=n;i++)
                a[i].x=read(),a[i].y=read(),a[i].pri=(double)a[i].x/a[i].y;

            sort(a+1,a+1+n,cmp);    // 按优先级降序排序

            ans=cntx[0]=cnty[n+1]=0;    // 初始化结果和前缀和数组

            // 计算前缀和: cntx[i]表示前i个节点的x值之和
            for(int i=1;i<=n;i++)
                cntx[i]=cntx[i-1]+a[i].x;

            // 计算后缀和: cnty[i]表示从第i个节点到末尾的y值之和
            for(int i=n;i>=1;i--)
                cnty[i]=cnty[i+1]+a[i].y;

            // 在特定范围内进行搜索
            for(int i=max((int)1,n/2-DELTA);i<=min(n/2+DELTA,n);i++)
                // 对每个中心点, 在其邻域内执行DFS搜索
                dfs(max((int)1,i-TEMP+1),min(n,i+TEMP-1),
                    cntx[max((int)0,i-TEMP)],cnty[min(n+1,i+TEMP)]);
        }
    }

```

```

        print(ans); // 输出结果
    }
    return 0;
}

```

4.3 雇佣问题

```

/*
// 单纯形法解对偶问题

read n, e // n: 天数, e: 雇佣方案数
read demand[1..n] // 每天需求
read each plan: [L, R, b] // 区间[L, R]可用, 费用 b[i]

// 约束矩阵 m[e][n], m[i][j] = 1 if j ∈ [L, R]
// 旋转操作: 选择第r行(basic变量)和第c列(free变量)进行交换
pivot(r, c):
    row[r] /= m[r][c] //归一化第r行
    row[i] -= m[i][c] * row[r] //消元其他行
    b[r] /= m[r][c]; //更新第r行的右端项

simplex():
    while True:
        c = max { demand[j] | j是非基变量 } // 入基
        if c <= 0: return OPTIMAL
        r = argmin { b[i]/m[i][c] | m[i][c]>0 } // 出基
        pivot(r, c) // 高斯消元交换基变量

MAIN():

    // 读取雇佣方案, 构建约束矩阵
    FOR i = 1 TO e DO:
        FOR j = L TO R DO:
            m[i][j] = 1.0 // 区间[L, R]内系数为1

    IF simplex() THEN
        return -b[0] // 输出最小费用(对偶关系)

时间复杂度: O(kne) k为pivot次数
空间复杂度: O(ne)
*/

#include <bits/stdc++.h>
#define N 1034 // 最大天数
#define E 10034 // 最大雇佣方案数
using namespace std;

const double eps = 1e-8; // 浮点精度

int n, e, l, r;
int i, j;

int id[N + E]; // 基变量和非基变量的编号
double m[E][N], b[E]; // m: 约束矩阵, b: 约束右端项(雇佣方案费用)
double *c = m[0]; // 目标函数系数(每天需求), 复用m[0]

```

```

double ans[N + E];          // 最终答案

// 旋转操作：选择第r行(basic变量)和第c列(free变量)进行交换
void pivot(int r, int c){
    int i, j;
    double coe = 1.0 / m[r][c]; // 归一化系数
    swap(id[n + r], id[c]);      // 交换基变量和非基变量的编号
    m[r][c] = 1.0;
    for(j = 1; j <= n; ++j)
        m[r][j] *= coe;        // 第r行归一化
    b[r] *= coe;
    // 高斯消元：更新其他行
    for(i = 0; i <= e; ++i){
        if(i == r) continue;
        coe = m[i][c];
        m[i][c] = 0.0;
        for(j = 1; j <= n; ++j)
            m[i][j] -= coe * m[r][j];
        b[i] -= coe * b[r];
    }
}

// 单纯形法主循环
bool simplex(){
    int i, j, basic, free;
    double G;
    for(;;){
        basic = free = 0;
        G = INFINITY;

        // 选择入基变量：找检验数最大的非基变量
        for(i = 1; i <= n; ++i)
            if(c[i] > c[free])
                free = i;

        if(!free) return true; // 所有检验数<=0，达到最优

        // 选择出基变量：找比率最小的基变量
        for(j = 1; j <= e; ++j)
            if(m[j][free] > eps && b[j] < G * m[j][free]){
                G = b[j] / m[j][free];
                basic = j;
            }

        if(!basic)
            return puts("Unbounded"), false; // 无界解

        pivot(basic, free); // 执行旋转操作
    }
}

int main(){
    scanf("%d%d", &n, &e); // n: 天数, e: 雇佣方案数

    // 读取每天的需求（对偶问题目标函数系数）

```

```
for(j = 1; j <= n; ++j)
    scanf("%lf", c + j);

// 读取雇佣方案，构建约束矩阵
for(i = 1; i <= e; ++i){
    scanf("%d%d%lf", &l, &r, b + i); // [l, r]时段，费用b[i]
    for(j = 1; j <= r; ++j)
        m[i][j] = 1.0; // 区间[l, r]内系数为1
}

if(simplex())
    printf("%.1f\n", -*b); // 输出最小费用(对偶关系)

return 0;
}
```